

화물이탈경고시스템

Cargo Monitoring System

설계 매뉴얼

3조(화물지킴이) | 강현서 · 성준하 · 이채원

2026 캡스톤 디자인 — 한형석 교수님

작성일: 2026년 5월

1. 프로젝트 개요_화물이탈경고시스템

1-1. 목표

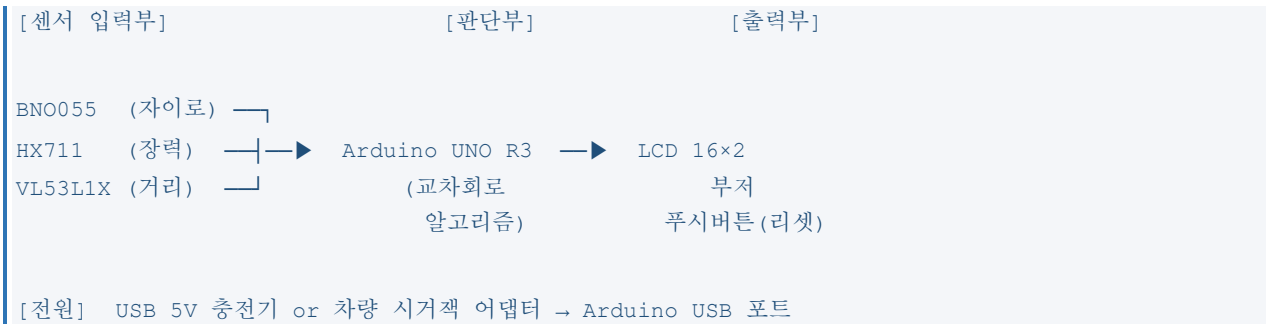
고속도로 낙하물 사고를 방지하기 위해 주행 중 화물 적재 상태를 실시간으로 감지하고, 위험 징후 발생 시 운전자에게 단계적으로 경보를 발령하는 시스템입니다.

1-2. 단계별 교차회로(Cross-Zone) 방식

소방 교차회로와 동일한 원리로 설계했습니다. 단일 센서 오작동에 의한 오경보를 방지하고, 복수 센서가 동시에 이상을 감지했을 때만 경보를 발령합니다.

단계	조건	출력 동작	의미
A — 주의 (모니터링)	센서 1개 임계값 초과	LCD 주의 표시 (부저 없음)	단순 노면 충격 가능성 — 주시 필요
B — 경고 (경고)	센서 2개 이상 동시 초과	LCD 경고 + 부저 간헐 경보	화물 쓸림 시작 — 감속 권고
C — 위험 (제동 권고)	센서 3개 모두 초과	LCD 위험 + 부저 연속 경보	화물 이탈 임박 — 즉시 정차

1-3. 시스템 구성도



💡 컴퓨터는 코드 업로드 시에만 필요합니다. 업로드 후에는 5V 전원만 있으면 독립 동작합니다.

2. 보유 부품 현황

발표자료 부품표 기준 — 현재 보유 중인 전체 부품입니다.

번호	부품명	수량	단가(원)	역할
1	Arduino UNO R3	1	7,000	메인 컨트롤러 — 모든 센서 판단 및 출력 제어
2	16×2 I2C LCD 모듈	2	1,500	상태 표시 (운전석 / 적재함 각 1개 활용 가능)
3	능동형 피에조 부저	3	1,500	단계별 경보음 출력
4	푸시 버튼	3	1,500	경보 리셋 입력
5	브레드보드	1	1,000	프로토타입 배선
6	VL53L1X 적외선 센서	5	12,000	화물 위치/거리 변화 감지
7	HX711 장력 센서 모듈	5	1,500	래싱벨트 장력 변화 감지
8	BNO055 자이로 센서	3	20,500	화물 기울기·쏠림 감지
9	점퍼 케이블 세트	5	3,200	센서-보드 연결
10	S형 로드셀	1	55,000	HX711과 연결, 실제 장력 측정

💡 보유 부품만으로 프로토타입 및 시제품 제작이 완전히 가능합니다. 라즈베리파이, 블루투스 모듈 없이도 독립 동작합니다.

3. 추가 권장 부품 및 확장 아이디어

3-1. 추가 권장 부품

부품명	가격(원)	우선순위	이유
HC-06 블루투스 모듈	3,000~5,000	★★★ 높음	스마트폰 연동 — 공모전 시연 임팩트 극대화
USB 5V 시거잭 어댑터	2,000~5,000	★★★ 높음	차량 탑재 시 독립 전원 공급 (필수)
방수 센서 케이스/박스	3,000~8,000	★★☆ 중간	야외 차량 환경 방수/방진 보호
ESP32 보드 (선택 교체)	10,000	★☆☆ 낮음	Wi-Fi+블루투스 내장, 향후 앱 고도화 시
점퍼 → 납땜 PCB 기판	2,000~5,000	★★☆ 중간	브레드보드 → 시제품 완성도 향상

⚠ 시거잭 어댑터는 차량 탑재 시제품에 필수입니다. 공모전 현장 시연 전 반드시 준비하세요.

3-2. 확장 아이디어 (단계별)

▶ 1단계 — 현재 목표 (6월 초)

- Arduino + 센서 3종 + LCD + 부저로 교차회로 경보 시스템 완성
- 모형 포터에 탑재 후 동작 확인

▶ 2단계 — 공모전 시제품 (6월-8월)

- 주행 데이터 누적 → 적재 패턴 분석 및 위험 예측
- 시거잭 어댑터로 차량 전원 독립 연결
- 화물 타입별 프리셋 임계값 자동 전환 기능(코드를 통해 자동으로 엑셀에 저장)
- 브레드보드 → PCB 기판 납땜으로 완성도 향상

▶ 3단계 — 향후 고도화 (발표자료 확장성 항목)

- ESP32로 교체 시 Wi-Fi 연동, 원격 모니터링 앱 구현 가능
- HC-06 블루투스 모듈 추가 → 스마트폰으로 센서 수치 실시간 확인 및 작동

💡 심화실습에서 라즈베리파이-폰 연동을 배운 직후 2단계에 바로 적용할 수 있습니다.

4. 핀 연결 설계

4-1. 전체 핀 연결 표

센서	센서 핀	Arduino 핀	통신방식	주의사항
BNO055	VIN / GND	5V / GND	I2C	ADR핀 → GND 필수 (주소 0x28)
BNO055	SDA / SCL	A4 / A5	I2C	I2C 버스 공유
VL53L1X	VIN / GND	3.3V / GND	I2C	⚠ 3.3V만! 5V 연결 시 파손
VL53L1X	SDA / SCL	A4 / A5	I2C	BNO055와 버스 공유
VL53L1X	XSHUT	D7	Digital	복수 센서 리셋 제어
HX711	VCC / GND	5V / GND	-	-
HX711	DT / SCK	D3 / D2	Digital	로드셀 4선 연결
LCD I2C	VCC / GND	5V / GND	I2C	주소 0x27 or 0x3F 확 인
LCD I2C	SDA / SCL	A4 / A5	I2C	I2C 버스 공유
부저	+ / -	D9 / GND	Digital	능동형 부저 사용
푸시버튼	핀1 / 핀2	D4 / GND	Digital	INPUT_PULLUP 설정

4-2. 핀 세부 연결 및 역할 설명

센서	센서 핀	Arduino 핀	역할	비고
BNO055	VIN	5V	전원 공급	
BNO055	GND	GND	접지	
BNO055	SDA	A4 (SDA)	I2C 데이터	I2C 버스
BNO055	SCL	A5 (SCL)	I2C 클럭	I2C 버스
VL53L1X	VIN	3.3V	전원 공급	3.3V 주의!
VL53L1X	GND	GND	접지	
VL53L1X	SDA	A4 (SDA)	I2C 데이터	BNO055와 공유
VL53L1X	SCL	A5 (SCL)	I2C 클럭	BNO055와 공유
VL53L1X	XSHUT	D7	리셋 제어	복수 센서시 필요
HX711	VCC	5V	전원 공급	
HX711	GND	GND	접지	
HX711	DT (DOUT)	D3	데이터 출력	디지털 핀
HX711	SCK	D2	클럭	디지털 핀
LCD (I2C)	VCC	5V	전원 공급	
LCD (I2C)	GND	GND	접지	
LCD (I2C)	SDA	A4 (SDA)	I2C 데이터	I2C 버스 공유
LCD (I2C)	SCL	A5 (SCL)	I2C 클럭	I2C 버스 공유
부저	+	D9	신호 출력	능동형 부저
부저	-	GND	접지	
푸시 버튼	한쪽 핀	D4	리셋 입력	INPUT_PULLUP
푸시 버튼	다른 핀	GND	접지	

4-3. I2C 주소 현황

기기	I2C 주소	비고
BNO055	0x28	ADR핀 GND 연결 시 — 반드시 이 설정 유지
VL53L1X	0x29	기본값 — BNO055 0x28과 충돌 없음
LCD	0x27 또는 0x3F	모듈 뒷면 점퍼로 확인

⚠ BNO055 ADR 핀을 반드시 GND에 연결하세요. VCC에 연결하면 주소가 0x29가 되어 VL53L1X와 충돌합니다.

4-4. 조립 순서 요약

1. 5V / 3.3V / GND 전원 레일 먼저 구성
2. I2C 공유 라인 연결 — SDA(A4), SCL(A5)에 3 개 기기 모두 연결
3. VL53L1X 는 반드시 3.3V 레일에 연결
4. HX711 + 로드셀 4 선 연결 후 D2/D3 에 연결
5. 부저 D9, 버튼 D4 연결
6. 라이브러리 설치 후 코드 업로드
7. 시리얼 모니터(9600 baud)로 센서 값 정상 출력 확인

4-5. 조립 및 테스트 순서

Step 1 — 브레드보드 기본 배선

- 5V 레일, 3.3V 레일, GND 레일 먼저 구성
- Arduino 의 5V → 5V 레일, 3.3V → 3.3V 레일, GND → GND 레일
- VL53L1X 는 반드시 3.3V 레일에 연결 (5V 인가 시 파손 주의!)

Step 2 — I2C 센서 연결

- BNO055 ADR 핀 → GND (주소 0x28 고정)
- SDA 라인: BNO055-SDA, VL53L1X-SDA, LCD-SDA 모두 A4 에 연결
- SCL 라인: BNO055-SCL, VL53L1X-SCL, LCD-SCL 모두 A5 에 연결
- I2C 스캐너 코드로 주소 충돌 확인 (Wire.scan 예제 사용)

Step 3 — HX711 + 로드셀 연결

- 로드셀 4 선(빨강/검정/초록/흰색) → HX711 모듈의 E+/E-/A+/A-에 연결
- HX711 DT → D3, SCK → D2

Step 4 — 부저 및 버튼

- 능동형 부저 + → D9, - → GND
- 푸시 버튼 한쪽 → D4, 다른 쪽 → GND (INPUT_PULLUP 사용)

Step 5 — 코드 업로드 및 검증

- 라이브러리 설치 완료 후 코드 붙여넣기
- 시리얼 모니터(9600 baud) 열고 Tilt / Tension / Dist 값 정상 출력 확인
- 화물 정상 상태: 모든 값 0 에 가까운지 확인
- 인위적으로 기울이기 → 단계별 경보 발동 테스트

5. 필수 라이브러리 설치

Arduino IDE → 툴 → 라이브러리 관리에서 아래 라이브러리를 설치하세요.

라이브러리	설치 이름	용도
Adafruit BNO055	Adafruit BNO055	자이로/IMU 데이터 읽기
Adafruit Unified Sensor	Adafruit Unified Sensor	BNO055 의존 라이브러리
VL53L1X	VL53L1X (pololu)	ToF 거리 센서
HX711	HX711 arduino library	로드셀 증폭 IC
LiquidCrystal I2C	LiquidCrystal I2C	I2C LCD 제어
Wire	내장 (기본 포함)	I2C 통신 (자동 포함)

6. Arduino 소스 코드

6-1. 전체 코드

아래 코드를 Arduino IDE에 붙여넣고 업로드하세요. 임계값은 3. 섹션 기준입니다.

```
// =====  
// 화물지킴이 시스템 - Arduino UNO R3  
// 센서: BNO055 (자이로) + HX711 (장력) + VL53L1X (거리)  
// 교차회로 3단계 경보 방식  
// =====  
  
#include <Wire.h>  
#include <Adafruit_Sensor.h>  
#include <Adafruit_BNO055.h>  
#include <VL53L1X.h>  
#include <HX711.h>  
#include <LiquidCrystal_I2C.h>  
  
// --- 핀 정의 ---  
#define HX711_DOUT    3  
#define HX711_SCK     2  
#define BUZZER_PIN    9  
#define BUTTON_PIN    4  
#define XSHUT_PIN     7  
  
// --- 임계값 설정 (테스트 후 조정) ---  
#define TILT_A        5.0    // A단계: 기울기 (도)
```

```

#define TILT_B      10.0    // B단계
#define TILT_C      15.0    // C단계
#define TENSION_A   0.10    // A단계: 장력 변화율 10%
#define TENSION_B   0.20    // B단계
#define TENSION_C   0.30    // C단계
#define DIST_A      50      // A단계: 거리 변화 (mm)
#define DIST_B      100     // B단계
#define DIST_C      200     // C단계

// — 객체 생성 —————
Adafruit_BNO055 bno = Adafruit_BNO055(55, 0x28);
VL53L1X vl53;
HX711 scale;
LiquidCrystal_I2C lcd(0x27, 16, 2);

// — 전역 변수 —————
float baseWeight = 0;      // 캘리브레이션 기준 장력
int   baseDist   = 0;      // 캘리브레이션 기준 거리
bool  alert      = false;  // 알람 상태

// =====
void setup() {
    Serial.begin(9600);
    Wire.begin();

    // 핀 모드
    pinMode(BUZZER_PIN, OUTPUT);
    pinMode(BUTTON_PIN, INPUT_PULLUP);
    pinMode(XSHUT_PIN, OUTPUT);

    // LCD 초기화
    lcd.init();
    lcd.backlight();
    lcd.print("HwamuL Guardian");

    // BNO055 초기화
    if (!bno.begin()) {
        lcd.setCursor(0,1); lcd.print("BNO055 ERR!");
        while(1);
    }
    bno.setExtCrystalUse(true);

    // VL53L1X 초기화 (XSHUT으로 리셋)
    digitalWrite(XSHUT_PIN, LOW);
    delay(10);
    digitalWrite(XSHUT_PIN, HIGH);
    delay(10);
    vl53.setTimeout(500);
    if (!vl53.init()) {
        lcd.setCursor(0,1); lcd.print("VL53 ERR!");
    }
}

```

```

    while(1);
}
vl53.setDistanceMode(VL53L1X::Long);
vl53.startContinuous(50);

// HX711 초기화
scale.begin(HX711_DOUT, HX711_SCK);
scale.set_scale(2280.f); // 보정값 - 캘리브레이션 필요
scale.tare();           // 0점 설정

// 기준값 저장 (캘리브레이션)
delay(1000);
baseWeight = scale.get_units(10);
baseDist   = vl53.read();
lcd.clear();
lcd.print("Ready!");
delay(1000);
}

// =====
void loop() {
    // 리셋 버튼 확인
    if (digitalRead(BUTTON_PIN) == LOW) {
        alert = false;
        noTone(BUZZER_PIN);
        lcd.clear(); lcd.print("RESET OK");
        delay(1000);
    }

    // ——— 센서 읽기 ———
    // 1) BNO055 기울기
    sensors_event_t event;
    bno.getEvent(&event);
    float tilt = abs(event.orientation.y); // 롤(쏠림) 각도

    // 2) HX711 장력 변화율
    float curWeight = scale.get_units(3);
    float tensionRatio = abs(curWeight - baseWeight) / (baseWeight + 0.001);

    // 3) VL53L1X 거리 변화
    int curDist = vl53.read();
    int distDiff = abs(curDist - baseDist);

    // ——— 교차회로 판단 ———
    // 각 센서별 단계 판정 (0=정상 1=A 2=B 3=C)
    int stTilt = (tilt >= TILT_C) ? 3 : (tilt >= TILT_B) ? 2 : (tilt >= TILT_A) ? 1 : 0;
    int stTension = (tensionRatio >= TENSION_C) ? 3 : (tensionRatio >= TENSION_B) ? 2 : (tensionRatio >= TENSION_A) ? 1 : 0;
    int stDist = (distDiff >= DIST_C) ? 3 : (distDiff >= DIST_B) ? 2 : (distDiff >= DIST_A) ? 1 : 0;

```

```

// 교차 조건: 활성화된 센서 수 세기
int triggered = (stTilt > 0 ? 1:0) + (stTension > 0 ? 1:0) + (stDist > 0 ? 1:0);
int maxStage = max(stTilt, max(stTension, stDist));

// 최종 경고 단계 결정
// C단계: 3개 모두 활성 OR 최대 단계가 3
// B단계: 2개 이상 활성 OR 최대 단계가 2
// A단계: 1개 활성
int alarmLevel = 0;
if (triggered == 3 || maxStage == 3) alarmLevel = 3;
else if (triggered >= 2 || maxStage == 2) alarmLevel = 2;
else if (triggered >= 1) alarmLevel = 1;

// — 출력 —————
lcd.clear();
if (alarmLevel == 0) {
    lcd.setCursor(0,0); lcd.print("STATUS: NORMAL");
    lcd.setCursor(0,1);
    lcd.print("T:"); lcd.print((int)tilt);
    lcd.print(" F:"); lcd.print((int)(tensionRatio*100));
    lcd.print("% D:"); lcd.print(distDiff/10);
    noTone(BUZZER_PIN);
}
else if (alarmLevel == 1) {
    lcd.setCursor(0,0); lcd.print("[A] CAUTION");
    lcd.setCursor(0,1); lcd.print("Check Cargo!");
    noTone(BUZZER_PIN);
}
else if (alarmLevel == 2) {
    lcd.setCursor(0,0); lcd.print("[B] WARNING!!");
    lcd.setCursor(0,1); lcd.print("Cargo Shifting");
    tone(BUZZER_PIN, 1000, 200); // 간헐 경고
    delay(500);
    noTone(BUZZER_PIN);
}
else if (alarmLevel == 3) {
    lcd.setCursor(0,0); lcd.print("[C] DANGER!!!");
    lcd.setCursor(0,1); lcd.print("SLOW DOWN NOW!");
    tone(BUZZER_PIN, 2000); // 연속 경고
}

// 시리얼 디버그 출력
Serial.print("Tilt:"); Serial.print(tilt);
Serial.print(" Tension:"); Serial.print(tensionRatio*100);
Serial.print("% Dist:"); Serial.print(distDiff);
Serial.print("mm Level:"); Serial.println(alarmLevel);

delay(200); // 200ms 주기
}

```

7. 경보 임계값 설계

아래는 초기 권장 임계값입니다. 6월 실험을 통해 실제 데이터 기반으로 반드시 보정해야 합니다.

단계	센서 조건	BNO055	HX711	VL53L1X
A (주의)	1개 이상 초과	기울기 > 5°	장력변화 > 10%	거리변화 > 5cm
B (경고)	2개 이상 동시 초과	기울기 > 10°	장력변화 > 20%	거리변화 > 10cm
C (위험)	3개 모두 초과	기울기 > 15°	장력변화 > 30%	거리변화 > 20cm
동작	LCD 표시	부저 간헐 경보	부저 연속 + LCD 위험 표시	동작

💡 임계값을 먼저 정하는 것이 아니라, 정상→위험 데이터를 수집한 뒤 그 사이에서 역산하는 방식으로 접근하세요.

8. 실험 및 임계값 도출 가이드

모형 포터를 활용한 단계별 실험 가이드입니다. 6월 중순 본격 실험 시 참고하세요.

8-1. 실험 4단계 절차

▶ Step 1 — 정상 상태 베이스라인 수집

- 각 적재물을 정상 상태로 올리고 평지에서 시리얼 모니터 값 기록
- 최소 3 분 이상 데이터 수집 — 이 값이 기준값(베이스라인)
- 시리얼 모니터 로그를 복사해 엑셀에 저장

▶ Step 2 — 인위적 위험 상황 재현

실험 상황	재현 방법	확인 포인트
급제동	모형 포터를 밀다가 갑자기 멈추기	BNO055 기울기 급변
급커브	포터를 좌우로 빠르게 기울이기	BNO055 롤값 변화
노면 충격	바퀴 아래 단차 통과 시뮬레이션	일시적 수치 스파이크
끈 이완	래싱벨트 의도적으로 살짝 풀기	HX711 장력 저하
화물 이탈 시작	박스를 손으로 살짝 밀어 위치 이동	VL53L1X 거리 변화

▶ Step 3 — 적재 타입별 반복 실험

적재 타입	위험 특성	주목 센서	실험 방법
박스류	무게중심 높음, 쓸림 취약	BNO055 + VL53L1X	좌우 기울이기, 급정지 시뮬레이션
파이프/원형	굴러갈 위험, 장력 급변	HX711 + VL53L1X	래싱벨트 느슨하게 후 기울이기
무거운 단일 화물	장력 크고 변화 둔함	HX711 위주	무게 증가 단계별 장력 측정
복합 적재	무게 편중, 복합 위험	3개 센서 전부	실제 적재 후 주행 시뮬레이션

▶ Step 4 — 임계값 역산

- 수집 데이터에서 정상 범위 최대값 → A 단계 임계값
- 위험 시작 시점 → B 단계 임계값
- 명백한 이탈 시점 → C 단계 임계값

8-2. 자동 데이터 저장 계획

6월 실험 시 Python 스크립트로 시리얼 데이터를 CSV 자동 저장하는 기능을 추가할 예정입니다.

- Arduino 시리얼 출력 → Python 스크립트 수신 → CSV 자동 기록
- 경보 발생 시점, 센서 수치, 타임스탬프 자동 저장
- 엑셀로 불러와 그래프 분석 → 임계값 보정에 활용

💡 '실험 데이터 기반으로 임계값을 도출했다'는 근거가 공모전 심사에서 신뢰도를 크게 높입니다.

9. 캘리브레이션 절차

9-1. HX711 (로드셀) 보정

- 알려진 무게(예: 1kg 추)를 로드셀에 올린 후 시리얼 모니터 실행
- `scale.get_units()` 출력값 확인
- 보정계수 = 실제무게(g) ÷ 출력값
- 코드의 `set_scale(2280.f)` 숫자를 보정계수로 교체 후 재업로드

9-2. VL53L1X 기준거리 보정

- 화물을 정상 적재 상태로 놓고 전원 인가
- `setup()` 내에서 자동으로 `baseDist` 저장됨 (별도 조작 불필요)
- 재적재 또는 적재물 변경 시 리셋 버튼(D4) 눌러 기준값 초기화

9-3. BNO055 초기 캘리브레이션

- 전원 인가 후 시리얼 모니터에서 Sys/Gyro/Accel/Mag 값 확인
- 모든 값이 3 이 될 때까지 센서를 8 자 형태로 천천히 회전
- 차량 수평 상태에서 캘리브레이션 완료 후 고정

⚠ BNO055 캘리브레이션은 전원을 껐다 켤 때마다 반복해야 합니다. 적재 전 매번 확인하세요.

10. 검증 체크리스트

프로토타입 동작 확인과 임계값 실험 두 단계로 나눠 진행합니다.

검증 항목	목표 기준	확인 방법
종강 전 — 전원 인가 후 센서 3종 정상 인식	시리얼 모니터 값 출력 확인	Arduino IDE 시리얼 모니터
종강 전 — LCD 화면 상태 표시 정상 출력	NORMAL 텍스트 표시	육안 확인
종강 전 — 부저 단계별 경보음 동작	A/B/C 단계 소리 구분	인위적 기울이기 테스트
종강 전 — 리셋 버튼 동작	경보 해제 확인	버튼 누름 후 NORMAL 복귀
종강 후 — 쉼림 검출 성공률	80% 이상	반복 실험 후 로그 분석
종강 후 — 정상 상태 오경보율	20% 이하	정상 주행 중 경보 발생 횟수
종강 후 — 연속 운전 안정성	5분 이상 오류 없음	장시간 동작 테스트

💡 내일은 '값이 나오는가'만 확인하면 됩니다. 임계값 정확도는 6월 실험 후 보정합니다.

공모전 파이팅! 🏆 3조 화물지킴이